Gamehok Tournament System Expansion Design

1. Introduction

1.1 Purpose of the Document

The purpose of this document is to propose a scalable and extensible tournament management system for the Gamehok platform that supports multiple head-to-head competitive tournament formats such as 1v1, 2v2, 4v4, and 5v5.

The proposed system extends the existing Battle Royale tournament structure into a generalized esports tournament engine capable of supporting multiple games, tournament styles, progression systems, and match management workflows.

1.2 Existing Platform Overview

The current Gamehok platform primarily supports Battle Royale-style tournaments where:

- Multiple teams participate in a single lobby
- Teams are ranked based on score and placement
- Top teams qualify to the next round
- Prize pools are distributed based on rankings

The platform already supports:

- Tournament registration
- Multi-round progression
- Squad participation
- Prize pools
- Schedules and organizers

However, the current architecture is optimized for Battle Royale formats and lacks support for direct head-to-head tournament systems.

1.3 Problem Statement

The current tournament engine does not support structured head-to-head competitive formats such as:

- 1v1
- 2v2
- Clash Squad formats
- 5v5 tournaments
- Knockout brackets
- League-based progression

Unlike Battle Royale tournaments, these formats require:

- Direct opponent pairing
- Team vs Team match management
- Bracket generation
- League/group handling
- Match progression logic
- Win/Loss-based qualification

A generalized tournament system is required to support these formats across multiple games.

1.4 Objectives

The proposed system aims to:

- Support multiple team sizes from 1v1 to 5v5
- Support Knockout, League, and Hybrid tournament formats
- Provide scalable tournament orchestration
- Support multiple esports games through a common tournament engine
- Enable configurable tournament structures
- Improve tournament engagement and flexibility
- Maintain extensibility for future tournament formats

2. Proposed Tournament System Overview

2.1 High-Level Idea

The proposed solution introduces a generalized esports tournament platform where users:

1. Select a game
2. Select a tournament style
3. Register individually or as a team
4. Participate in tournaments managed by the platform

The platform itself does not run the actual gameplay. Instead, it manages:

- Tournament lifecycle
- Team registration
- Match progression
- Bracket generation
- League standings
- Qualification flow
- Prize distribution

2.2 Supported Match Formats

The platform will support:

Match Format	Description
1v1	Single-player teams competing directly
2v2	Two-player teams
3v3	Three-player teams
4v4	Four-player teams
5v5	Full team-based competitive mode

To simplify backend architecture, every participant is treated as a Team entity.

Examples:

- 1v1 → Team with one player
- 2v2 → Team with two players
- 5v5 → Team with five players

This creates a unified tournament model where every match is handled as:

Team A vs Team B

regardless of player count.

2.3 Tournament Types

The system will support multiple tournament styles.

2.3.1 Quick Match Mode

Quick Match mode is designed for casual users who want immediate gameplay.

Flow:

- Player/team enters matchmaking queue
- Platform waits for opponent
- Match starts immediately after pairing
- Winner/score is recorded
- Session ends

Features:

- No tournament progression
- No brackets
- No league tables

- Lightweight matchmaking
- Fast gameplay experience

2.3.2 Knockout Tournament Mode

Knockout tournaments are elimination-based.

Flow:

- Teams register before deadline
- Bracket is generated
- Teams compete head-to-head
- Losing team is eliminated
- Winning team advances to next round

Example:

Quarter Final → Semi Final → Final

This format is suitable for:

- Fast tournaments
- Casual competitions
- Weekend esports events

2.3.3 League Tournament Mode

League tournaments focus on long-format competition.

Flow:

- Teams are divided into groups
- Teams inside groups play each other
- Points table is maintained
- Top teams qualify
- Qualified teams may enter playoffs/finals

Features:

- Multiple matches per team
- Standings and rankings
- Better competitive balance
- Longer engagement duration

2.3.4 Hybrid Tournament Mode

Hybrid tournaments combine league stages with knockout playoffs.

Example:

Stage 1:

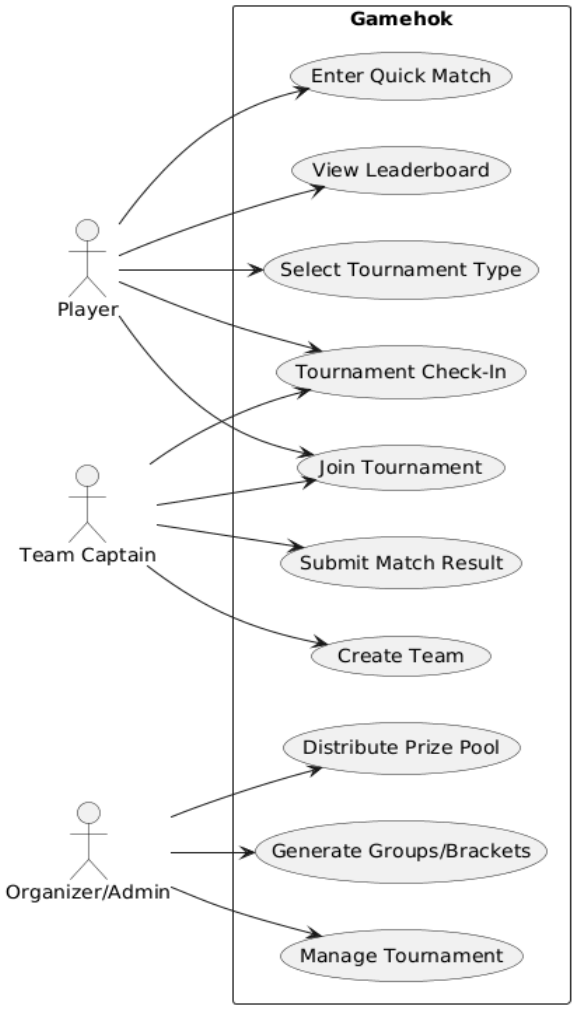
- Multiple groups
- Round robin league matches
- Top teams qualify

Stage 2:

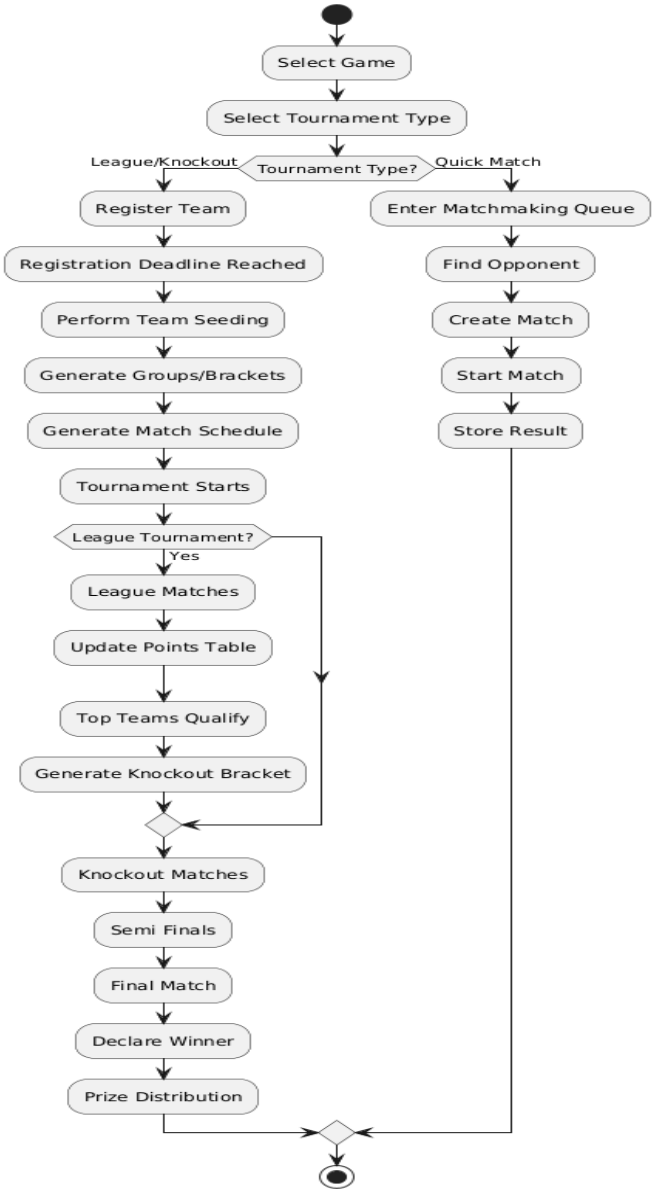
- Qualified teams enter knockout bracket
- Semi Finals and Finals conducted

This format is commonly used in large esports tournaments.

Use Case Diagram:



Tournament Activity Diagram:



Hybrid Tournament Flow:

3. Team Formation System

3.1 Team Creation

Users can:

- Create their own teams
- Invite players
- Join as pre-made squads

Each team has:

- Team ID
- Team name
- Captain
- Players
- Match format compatibility

3.2 Solo Queue Support

The system supports solo queue participation.

Example:

- User selects 2v2 tournament
- User enters alone
- Matchmaking system automatically pairs the user with another solo player
- Temporary team is created

This improves accessibility for users who do not already have teammates.

3.3 Team Validation Rules

Before tournament entry:

- Team size must match tournament format
- All players must confirm participation
- Duplicate registrations must be prevented

Example:

- 2v2 tournament requires exactly 2 players per team
- 5v5 tournament requires exactly 5 players per team

4. Tournament Registration and Scheduling

4.1 Tournament Registration Window

League and Knockout tournaments are scheduled events.

Example:

- Tournament Start Time: 9:00 PM
- Registration Deadline: 8:30 PM

The registration deadline is required to:

- Finalize participating teams
- Generate balanced groups/brackets
- Perform seeding
- Create schedules
- Prevent last-minute structural changes

4.2 Team Seeding and Distribution

After registration closes, the platform performs tournament seeding.

Goal:

- Avoid placing all strong teams in the same group
- Maintain balanced competition

Possible seeding factors:

- Previous tournament performance
- Platform rankings
- Win rates
- Tournament history

Teams are distributed evenly across groups/brackets.

Example:

Instead of:

Group A → Strong teams only

The system distributes:

- Strong teams
- Medium teams
- New teams

across all groups.

5. Group and League System

5.1 Group Generation

Tournament organizers/admins can configure:

- Number of groups
- Qualification count
- Tournament stages

Example:

20 teams:

- 5 groups
- 4 teams per group

5.2 Round Robin Match Generation

Inside each group:

- Every team plays against all other teams
- Points are awarded based on match results

Example:

Win → 2 points Loss → 0 points

5.3 Qualification Logic

After group matches:

- Top teams qualify
- Remaining teams are eliminated

Example:

- Top 2 teams from each group qualify
- Qualified teams enter playoffs

5.4 Tie-Breaker Logic

If teams have equal points, tie-breakers are used.

Possible tie-breakers:

- Head-to-head result
- Total kills
- Round difference
- Win percentage

Tie-breaker logic should remain configurable because different games may use different competitive rules.

6. Knockout System

6.1 Bracket Generation

After registration or qualification:

- Platform generates tournament brackets
- Teams are paired automatically

Example:

Team A vs Team B Team C vs Team D

6.2 Winner Progression

After each match:

- Winner advances
- Loser is eliminated

Example:

Quarter Final → Semi Final → Final

6.3 Bye Round Handling

If participant count is uneven:

- Certain teams may receive automatic advancement (bye rounds)

Example:

5 teams in an 8-team bracket.

7. Match Management System

7.1 Match Lifecycle

Each match follows:

1. Match creation
2. Team assignment
3. Match scheduling
4. Match start
5. Result submission
6. Validation
7. Progression update

7.2 Match Result Handling

After gameplay completion:

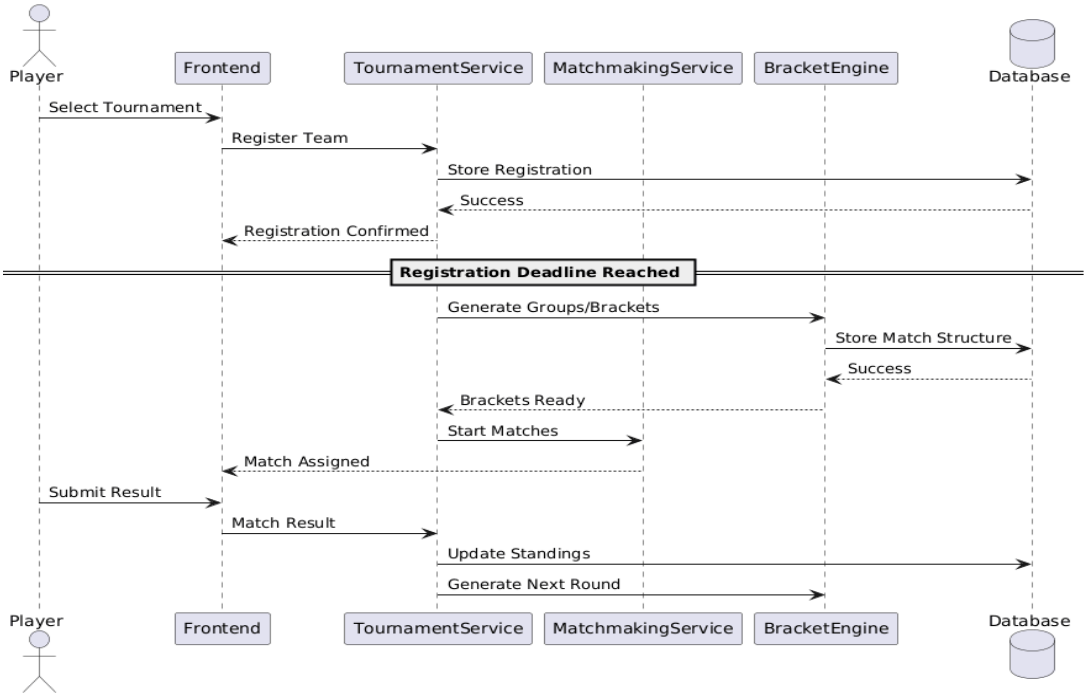
- Winning team is recorded
- Scores/kills are stored
- Tournament standings are updated
- Next-round progression is generated

7.3 Match Verification

Since Gamehok manages tournaments externally from the actual games, match verification may include:

- Admin approval
- Screenshot submission
- Organizer verification
- API integrations (future scope)

Tournament Registration and Match Sequence Diagram:



8. Prize Pool and Rewards

8.1 Entry Fee System

Certain tournaments may include entry fees.

Example:

- Each team pays entry fee
- Platform creates tournament prize pool
- Winners receive rewards

8.2 Prize Distribution

Prize distribution may be based on:

- Final rankings
- Tournament stage reached
- Winning position

Example:

1st Place → 60% 2nd Place → 30% 3rd Place → 10%

8.3 Future Engagement Enhancements

Large tournaments may include dynamic prize pool mechanics.

Example:

- Prize pool increases as tournament progresses
- Eliminations contribute to reward scaling
- Players remain motivated for long-format tournaments

This creates stronger player engagement and long-term participation incentives.

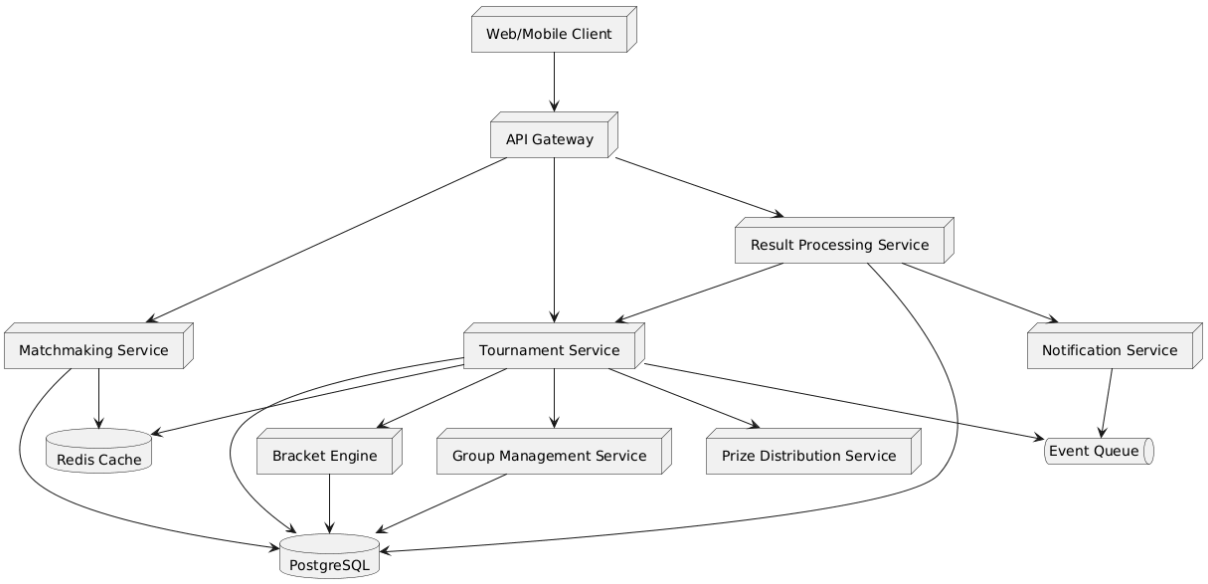
9. System Architecture

9.1 Core Modules

The system can be divided into multiple modules:

- Tournament Engine
- Matchmaking Service
- Group Management Service
- Bracket Management Service
- Match Result Service
- Notification Service
- Prize Distribution Service

System Architecture Diagram:



9.2 Tournament Engine

The tournament engine is responsible for:

- Tournament lifecycle management
- Progression logic
- Group generation
- Qualification handling
- Stage transitions

This engine should remain configurable and extensible.

9.3 Matchmaking System

Quick Match mode uses lightweight matchmaking.

Flow:

Player/team enters queue → Opponent found → Match created → Match starts

10. Database Design

10.1 Main Entities

Key entities include:

- Player
- Team
- Tournament
- TournamentStage
- Group
- Match
- MatchParticipant
- MatchResult
- Leaderboard

10.2 Unified Team Model

The platform uses a unified Team-based architecture.

Benefits:

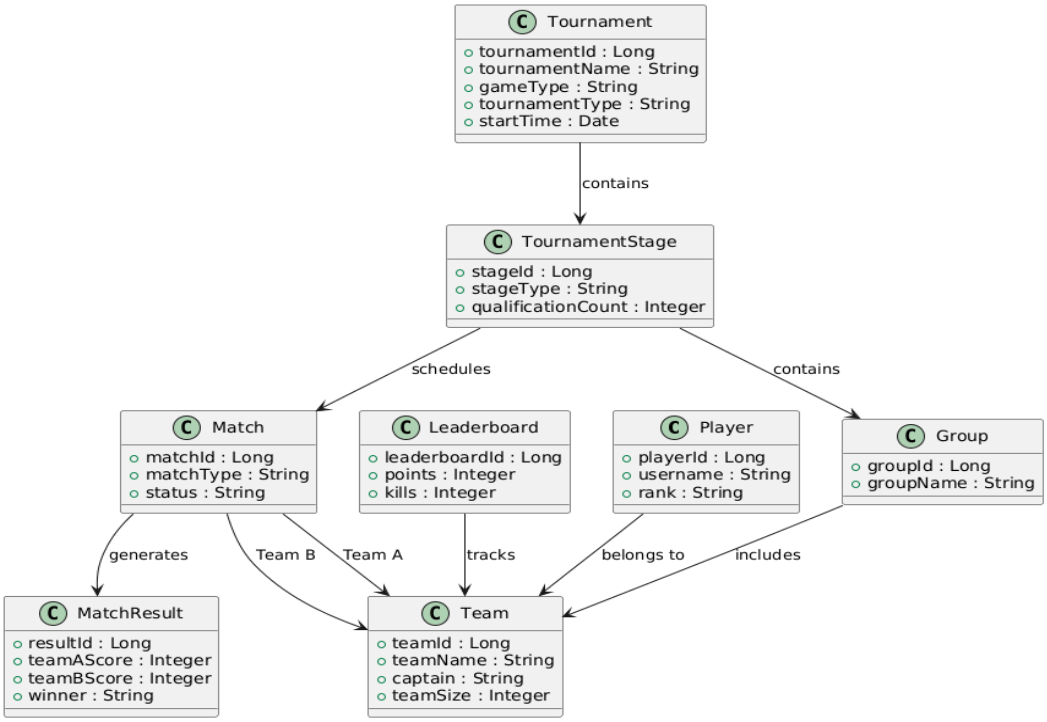
- Simplifies tournament logic
- Simplifies matchmaking
- Common match structure across all modes
- Easier scalability

All matches are represented as:

Team A vs Team B

regardless of team size.

Tournament Class Diagram:



11. Non-Functional Requirements

11.1 Scalability

The platform should support:

- Multiple simultaneous tournaments
- Thousands of teams
- Multiple games
- Real-time updates

11.2 Extensibility

The architecture should support future additions such as:

- Double elimination tournaments
- AI matchmaking
- Cross-game tournaments
- Advanced analytics

11.3 Reliability

The platform should ensure:

- Accurate progression
- Consistent standings
- Fault-tolerant match updates
- Stable tournament lifecycle management

12. Conclusion

The proposed tournament system transforms the existing Battle Royale-focused structure into a generalized esports tournament platform capable of supporting multiple games and tournament styles.

The design introduces:

- Flexible tournament formats
- League and knockout systems
- Team-based matchmaking
- Scalable progression handling
- Extensible tournament orchestration

The proposed architecture prioritizes:

- Simplicity for MVP implementation
- Future extensibility
- Competitive fairness
- User engagement
- Scalable tournament management

This system creates a foundation for supporting diverse esports competitions on the Gamehok platform.